

Tuyển Tập Bài Tập: Kỹ Thuật Xử Lý Ngoại Tuyến & Quét Sự Kiện (Offline Processing & Sweep-Line)

(Curated Competitive Programming Problems on Offline Queries, Mo's Algorithm & Dynamic Connectivity)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)
Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool
Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng Tổng Hợp 16 Bài Tập Tuyển Chọn

STT	Mã Bài	Tên Bài Toán	Nguồn	Độ Khó	Chủ Đề Trọng Tâm
1	CSES 1734	Distinct Values Queries	CSES	☆☆☆	Quét cận phải R tăng dần + Fenwick Tree $O((N + Q) \log N)$
2	SPOJ DQUERY	D-query	SPOJ	☆☆☆	Đếm phần tử phân biệt bằng Thuật toán Mo hoặc Quét cận phải
3	CF 220B	Little Elephant and Array	Codeforces	☆☆☆	Mo's algorithm đếm các số có tần số bằng đúng giá trị của nó
4	CSES 2169	Nested Ranges Count	CSES	☆☆☆	Sắp xếp đoạn 2 tiêu chí + Quét Offline với Fenwick Tree
5	CF 617E	XOR and Favorite Number	Codeforces	☆☆☆☆	Mo's algorithm trên mảng tiền tố XOR, duy trì tần số $X \oplus K$
6	CF 86D	Powerful array	Codeforces	☆☆☆☆	Mo's algorithm tính $\sum cnt[x]^2 \cdot x$ với $N, Q = 2 \times 10^5$
7	CF 1213G	Path Queries	Codeforces	☆☆☆☆	Sắp xếp cạnh và truy vấn theo trọng số tăng dần + DSU
8	CF 1405E	Fixed Point Removal	Codeforces	☆☆☆☆	Quét cận phải R kết hợp Walk on Segment Tree
9	CF 1513D	GCD and MST	Codeforces	☆☆☆☆	Sắp xếp phần tử theo giá trị tăng dần, mở rộng đoạn GCD + DSU
10	CF 522D	Closest Equals	Codeforces	☆☆☆☆	Quét cận phải R + Segment Tree RMQ khoảng cách 2 số trùng nhau

STT	Mã Bài	Tên Bài Toán	Nguồn	Độ Khó	Chủ Đề Trọng Tâm
11	CF 940F	Machine Learning	Codeforces	★★★★★	Mo's algorithm có thao tác cập nhật (Mo with Updates) $O(N^{5/3})$
12	CF 1117G	Recursive Queries	Codeforces	★★★★★	Quét Offline 2 chiều trái & phải + Fenwick Tree Range Add
13	SPOJ METEORS	Metereors (POI 2011)	SPOJ / POI	★★★★★	Chặt nhị phân song song (Parallel Binary Search)
14	CF 1217F	Forced Online Graph	Codeforces	★★★★★	So sánh Đồ thị động Offline vs Chặn Online theo khối $\sqrt{M}$
15	CF 576E	Painting Edges	Codeforces	★★★★★	Dynamic Connectivity trên K màu độc lập, Parity Rollback DSU
16	CF 1446D2	Frequency Problem (Hard)	Codeforces	★★★★★	Chia căn tần số kết hợp quét tiền tố cân bằng

💡 Phân Tích & Hướng Dẫn Chi Tiết (3-Tier Hints)

Bài 1: CSES 1734 — Distinct Values Queries

- **Đề bài:** Cho mảng A gồm N số nguyên ($N, Q \leq 2 \times 10^5$). Trả lời Q truy vấn đếm số lượng giá trị phân biệt trong đoạn $[L, R]$.
- **Gợi ý 1 (Nhận xét bản chất):**
 - Trong tiền tố $[1, R]$, chỉ có lần xuất hiện cuối cùng của mỗi giá trị là có ý nghĩa với các đoạn $[L, R]$.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Sắp xếp các truy vấn theo cận phải R tăng dần.
 - Dùng bảng băm `last_pos[v]` lưu vị trí xuất hiện gần nhất của giá trị v .
 - Khi duyệt tới R : Nếu $A[R]$ đã từng xuất hiện tại p , trừ 1 tại p trên Fenwick Tree (`bit.add(p, -1)`). Sau đó cộng 1 tại R (`bit.add(R, +1)`).
 - Với mọi truy vấn có cận phải là R , đáp án là `bit.query(L, R)`.
- **Gợi ý 3 (Độ phức tạp & Cạm bẫy):**
 - Thời gian: $O((N + Q) \log N)$, Bộ nhớ: $O(N + Q)$.
 - Cạm bẫy: Cần nén giá trị mảng A nếu dùng mảng tĩnh thay cho `unordered_map`.

Bài 2: SPOJ DQUERY — D-query

- **Đề bài:** Cho mảng A gồm N phần tử ($N \leq 30000, Q \leq 200000$). Đếm số phần tử phân biệt trong đoạn $[L, R]$.
- **Gợi ý 1 (Nhận xét bản chất):**

- Bài toán kinh điển có thể giải bằng cả 2 cách: Quét cận phải $O((N + Q) \log N)$ hoặc Mo's Algorithm $O((N + Q)\sqrt{N})$.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Nếu dùng Mo's algorithm: Duy trì mảng tần số `cnt[v]`.
- Khi thêm phần tử: Nếu `++cnt[val] == 1`, tăng biến đếm `distinct++`.
- Khi xóa phần tử: Nếu `--cnt[val] == 0`, giảm biến đếm `distinct--`.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(N\sqrt{Q})$ với Mo, hoặc $O((N + Q) \log N)$ với Sweep-line.

Bài 3: Codeforces 220B — Little Elephant and Array

- **Đề bài:** Cho mảng A ($N, Q \leq 10^5$). Đoạn $[L, R]$ được gọi là đẹp nếu tồn tại các số x sao cho x xuất hiện đúng x lần trong đoạn. Đếm số lượng số thỏa mãn điều kiện đó trong mỗi truy vấn.
- **Gợi ý 1 (Nhận xét bản chất):**
- Nhận xét số học: Một số $x > N$ không bao giờ có thể xuất hiện x lần trong mảng.
- Số lượng giá trị $x \leq N$ có thể xuất hiện x lần là không quá $\sqrt{2N} \approx 450$ số (vì $1 + 2 + \dots + k \leq N$).
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Áp dụng Mo's algorithm: Duy trì mảng tần số `freq[v]`.
- Khi `freq[v]` chuyển từ $x - 1 \rightarrow x$, tăng đáp án lên 1.
- Khi `freq[v]` chuyển từ $x \rightarrow x + 1$, giảm đáp án đi 1.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(N\sqrt{Q})$, Bộ nhớ: $O(N)$.

Bài 4: CSES 2169 — Nested Ranges Count

- **Đề bài:** Cho N đoạn $[x_i, y_i]$. Với mỗi đoạn, hãy tính: (1) Số lượng đoạn khác chứa nó, (2) Số lượng đoạn khác bị nó chứa.
- **Gợi ý 1 (Nhận xét bản chất):**
- Đoạn i chứa đoạn j khi $x_i \leq x_j$ và $y_i \geq y_j$.
- Đây là bài toán 2 chiều (x, y) . Ta khử 1 chiều bằng cách sắp xếp!
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Sắp xếp các đoạn theo x tăng dần. Nếu x bằng nhau, sắp xếp theo y giảm dần.
- Nén tọa độ các giá trị y .
- Duyệt tuần tự các đoạn: Chiều x đã được thỏa mãn tự động. Chiều y được quản lý bằng Fenwick Tree.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(N \log N)$, Bộ nhớ: $O(N)$.
- Cận bậc: Xử lý các đoạn trùng nhau hoàn toàn: Tiêu chí sắp xếp y giảm dần đảm bảo đoạn lớn hơn được xử lý trước.

Bài 5: Codeforces 617E — XOR and Favorite Number

- **Đề bài:** Cho mảng A và số K . Cho Q truy vấn $[L, R]$, đếm số cặp (i, j) với $L \leq i \leq j \leq R$ sao cho $A_i \oplus A_{i+1} \cdots \oplus A_j = K$.
- **Gợi ý 1 (Nhận xét bản chất):**
- Đặt P là mảng tiền tố XOR: $P_i = A_1 \oplus \cdots \oplus A_i$ (với $P_0 = 0$).
- Đoạn $A[i \dots j]$ có XOR bằng K khi và chỉ khi:

$$P_j \oplus P_{i-1} = K \iff P_{i-1} = P_j \oplus K$$

- Bài toán quy về: Trên mảng P , đếm số cặp (u, v) với $L - 1 \leq u < v \leq R$ sao cho $P_u \oplus P_v = K$.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Áp dụng Mo's algorithm trên các đoạn $[L - 1, R]$.
- Duy trì mảng đếm tần số `cnt[val]`.
- Khi thêm phần tử $x = P_{idx}$: Cộng vào đáp án `cnt[x ^ K]`, sau đó tăng `cnt[x]++`.
- Khi xóa phần tử $x = P_{idx}$: Giảm `cnt[x]--`, sau đó trừ khỏi đáp án `cnt[x ^ K]`.
- **Gợi ý 3 (Độ phức tạp & Cận bẫy):**
- Thời gian: $O((N + Q)\sqrt{N})$, Bộ nhớ: $O(N + 2^{20})$.
- Cận bẫy: Cận trái của Mo phải là $L - 1$. Đáp án có thể lên tới $O(Q \cdot N^2) \approx 10^{10}$, bắt buộc dùng `long long`.

Bài 6: Codeforces 86D — Powerful array

- **Đề bài:** Cho mảng A ($N, Q \leq 2 \times 10^5, A_i \leq 10^6$). Sức mạnh của một đoạn được định nghĩa là $\sum_x K_x^2 \cdot x$, trong đó K_x là số lần xuất hiện của giá trị x trong đoạn. Tính sức mạnh của mỗi truy vấn $[L, R]$.
- **Gợi ý 1 (Nhận xét bản chất):**
- Khi tần số của x tăng từ $K_x \rightarrow K_x + 1$, phần đóng góp của x thay đổi:

$$\Delta = ((K_x + 1)^2 - K_x^2) \cdot x = (2K_x + 1) \cdot x$$

- Khi tần số giảm từ $K_x \rightarrow K_x - 1$:

$$\Delta = ((K_x - 1)^2 - K_x^2) \cdot x = -(2K_x - 1) \cdot x$$

- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Dùng Mo's Algorithm với kích thước khối $B = \lceil N/\sqrt{Q} \rceil \approx 450$.
- Cài đặt thêm/xóa trong $O(1)$.
- Áp dụng kỹ thuật Odd-Even Sort để tránh TLE với giới hạn thời gian chặt.
- **Gợi ý 3 (Độ phức tạp & Cận bẫy):**
- Thời gian: $O(N\sqrt{Q})$, Bộ nhớ: $O(N + \max A)$.
- Cận bẫy: Dùng kiểu `long long` cho biến lưu tổng sức mạnh.

Bài 7: Codeforces 1213G — Path Queries

- **Đề bài:** Cho cây có trọng số N đỉnh. Có Q truy vấn w : Đếm số cặp đỉnh (u, v) ($u < v$) sao cho trọng số lớn nhất của cạnh trên đường đi giữa u và v không vượt quá w .
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Nếu coi cây rỗng ban đầu và lần lượt thêm các cạnh có trọng số $\leq w$, hai đỉnh u, v thỏa mãn điều kiện khi và chỉ khi chúng thuộc cùng một thành phần liên thông.
 - Một thành phần liên thông kích thước S đóng góp $\binom{S}{2} = \frac{S(S-1)}{2}$ cặp đỉnh hợp lệ.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Sắp xếp tất cả $N - 1$ cạnh theo trọng số tăng dần.
 - Sắp xếp Q truy vấn theo giá trị w tăng dần.
 - Dùng con trỏ duyệt song song: Thêm các cạnh có trọng số $\leq w$ vào DSU. Khi gộp hai thành phần S_u và S_v , số cặp hợp lệ tăng thêm $S_u \cdot S_v$.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O((N + Q) \log(N + Q))$, Bộ nhớ: $O(N + Q)$.
-

Bài 8: Codeforces 1405E — Fixed Point Removal

- **Đề bài:** Cho mảng A . Phần tử $A[i]$ là điểm bất động nếu $A[i] = i$. Mỗi bước được xóa một điểm bất động. Cho Q truy vấn $[x, y]$: Nếu xóa x phần tử đầu và y phần tử cuối, đếm số điểm bất động tối đa có thể xóa.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Đoạn còn lại sau khi xóa tiền tố x và hậu tố y là đoạn con $[x + 1, N - y]$.
 - Để một phần tử tại vị trí i bị xóa, cần có ít nhất $k = i - A[i]$ phần tử đứng trước nó đã bị xóa.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Sắp xếp truy vấn theo cận phải $R = N - y$.
 - Duyệt i từ 1 đến N : Dùng Walk on Segment Tree tìm vị trí L lớn nhất sao cho tiền tố $[1, L]$ đã có ít nhất k phần tử bị xóa. Sau đó cộng 1 vào đoạn $[1, L]$.
 - Truy vấn tại R lấy giá trị tại $L = x + 1$.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O((N + Q) \log N)$, Bộ nhớ: $O(4N)$.
-

Bài 9: Codeforces 1513D — GCD and MST

- **Đề bài:** Cho mảng A . Ta có thể thêm cạnh giữa i và j với trọng số A_k nếu $\gcd(A_i, \dots, A_j) = A_k = \min(A_i \dots A_j)$. Ngoài ra luôn có thể thêm cạnh giữa hai đỉnh kề nhau với trọng số P . Tìm cây khung nhỏ nhất (MST).
- **Gợi ý 1 (Nhận xét bản chất):**
 - Áp dụng tư duy Kruskal: Luôn ưu tiên dùng các cạnh có trọng số nhỏ hơn P .
 - Cạnh rẻ nhất được tạo bởi các phần tử A_i nhỏ nhất.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Sắp xếp các đỉnh i theo giá trị A_i tăng dần.

- Với mỗi đỉnh i có $A_i < P$: Mở rộng sang trái chừng nào $A_l \bmod A_i == 0$, mở rộng sang phải chừng nào $A_r \bmod A_i == 0$.
- Nối các đỉnh liên tiếp trong đoạn mở rộng bằng DSU. Dừng mở rộng ngay khi gặp một đỉnh đã được nối với i .
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(N \log N)$, Bộ nhớ: $O(N)$.

Bài 10: Codeforces 522D — Closest Equals

- **Đề bài:** Cho mảng A . Cho Q truy vấn $[L, R]$. Tìm khoảng cách nhỏ nhất giữa hai phần tử bằng nhau trong đoạn: $\min\{y - x \mid L \leq x < y \leq R, A[x] = A[y]\}$.
- **Gợi ý 1 (Nhận xét bản chất):**
- Quét cận phải R từ 1 đến N .
- Mỗi khi gặp $A[R]$ có lần xuất hiện trước đó tại $p = \text{prev}[A[R]]$, ta có một cặp hợp lệ bắt đầu tại p với khoảng cách $R - p$.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Dựng Segment Tree RMQ trên các vị trí bắt đầu $x \in [1, N]$.
- Khi duyệt tới R : Cập nhật tại vị trí p : `st.update(p, R - p)`.
- Với các truy vấn có cận phải bằng R , đáp án là `st.query(L, R)`.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O((N + Q) \log N)$, Bộ nhớ: $O(4N)$.

Bài 11: Codeforces 940F — Machine Learning

- **Đề bài:** Cho mảng A . Hỗ trợ 2 loại thao tác: (1) Truy vấn $[L, R]$ tìm MEX của tập các tần số xuất hiện của các phần tử trong đoạn, (2) Cập nhật điểm $A[p] = x$.
- **Gợi ý 1 (Nhận xét bản chất):**
- Bài toán có cập nhật điểm và truy vấn đoạn $\implies$ Sử dụng **Mo's Algorithm with Updates (Mo 3D)**.
- Mỗi truy vấn được biểu diễn bằng bộ 3: (L, R, T) với T là số thao tác cập nhật đã xảy ra trước truy vấn này.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Kích thước khối tối ưu: $B = N^{2/3}$.
- Sắp xếp theo $(L/B, R/B, T)$.
- Di chuyển 3 con trỏ `curL`, `curR`, và `curT` (thực hiện hoặc hoàn tác cập nhật).
- Độ phức tạp tổng cộng: $O(N^{5/3})$.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(N^{5/3})$, Bộ nhớ: $O(N)$.
- Cận bậc: Cần cập nhật cả giá trị trong mảng A khi con trỏ T di chuyển.

Bài 12: Codeforces 1117G — Recursive Queries

- **Đề bài:** Cho hoán vị P . Cho Q truy vấn $[L, R]$. Tính tổng kích thước các cây con trong Cartesian Tree sinh bởi đoạn $[L, R]$.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Mỗi phần tử i đóng góp $\min(R, r_i - 1) - \max(L, l_i + 1) + 1$ vào kết quả.
 - Tách thành 2 thành phần độc lập: Đóng góp bên trái và đóng góp bên phải.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Quét 2 lượt Offline: Lượt 1 quét từ trái sang phải với các truy vấn R ; Lượt 2 quét từ phải sang trái với các truy vấn L .
 - Sử dụng Fenwick Tree hỗ trợ Range Add và Range Sum.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O((N + Q) \log N)$, Bộ nhớ: $O(N + Q)$.
-

Bài 13: SPOJ METEORS — Meteors (POI 2011)

- **Đề bài:** Có N quốc gia và M trạm vũ trụ. Mỗi quốc gia cần thu thập P_i mẫu thiên thạch. Có K đợt mưa sao băng, mỗi đợt rải thiên thạch lên một cung tròn $[l, r]$ với số lượng a . Tìm đợt mưa sao băng sớm nhất mà mỗi quốc gia đạt chỉ tiêu.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Bài toán kinh điển số 1 về **Chặt nhị phân song song (Parallel Binary Search)**.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Hàm `solve(Q_list, L, R)`: Tập các quốc gia đang có đáp án nằm trong khoảng đợt mưa $[L, R]$.
 - Đặt $mid = (L + R) / 2$. Dùng Fenwick Tree rải các đợt mưa từ $L \dots mid$.
 - Với mỗi quốc gia trong nhóm: Tính tổng số thiên thạch thu được. Nếu $\geq P_i$, đưa vào nhóm trái $[L, mid]$; ngược lại trừ bớt thiên thạch và đưa vào nhóm phải $[mid + 1, R]$.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O((N + M + K) \log M \log K)$.
 - Cận bậc: Cung tròn có thể đi qua vị trí 1 ($l > r$). Tràn số khi cộng dồn thiên thạch (chặn tối đa P_i).
-

Bài 14: Codeforces 1217F — Forced Online Graph

- **Đề bài:** Cho đồ thị động. Hỗ trợ thêm/xóa cạnh và hỏi tính liên thông. Đề bài mã hóa truy vấn ép phải giải Online: Các đỉnh được XOR với kết quả của truy vấn trước.
- **Gợi ý 1 (Nhận xét bản chất):**
 - Vì bị ép Online, ta không thể dùng Segment Tree trên thời gian thông thường.
 - Tuy nhiên, mỗi truy vấn chỉ có 2 khả năng cạnh: (x, y) hoặc $((x \bmod n) + 1, (y \bmod n) + 1)$.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Kỹ thuật chia khối trên thao tác (Block Decomposition / Rebuilding): Chia M thao tác thành các khối kích thước $B \approx \sqrt{M}$.
 - Sau mỗi B thao tác, ta dựng lại đồ thị nền và co các thành phần liên thông bằng DSU.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O(M \sqrt{M} \log N)$.

Bài 15: Codeforces 576E — Painting Edges

- **Đề bài:** Cho đồ thị N đỉnh, M cạnh và K màu. Có Q truy vấn: Sơn cạnh e bằng màu c . Thao tác sơn chỉ được chấp nhận nếu sau khi sơn, đồ thị con tạo bởi các cạnh màu c vẫn là **đồ thị hai phía (Bipartite Graph)**. Nếu không được chấp nhận, cạnh giữ nguyên màu cũ.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Mỗi màu là một đồ thị hai phía độc lập.
 - Cần hỗ trợ kiểm tra tính hai phía khi thêm/xóa cạnh $\implies$ Dùng **Parity Rollback DSU** (DSU thể vị khoảng cách chẵn/lẻ).
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Chia để trị theo thời gian (Time D&C): Khi một cạnh được sơn màu c thành công tại thời điểm t , khoảng tồn tại của màu đó kéo dài tới lần sơn tiếp theo của cạnh đó.
 - Nếu thao tác sơn thất bại, khôi phục lại màu cũ.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O(Q \log Q \log N)$, Bộ nhớ: $O(Q \log Q + M)$.
-

Bài 16: Codeforces 1446D2 — Frequency Problem (Hard)

- **Đề bài:** Cho mảng A gồm N phần tử ($N \leq 2 \times 10^5$). Tìm độ dài đoạn con dài nhất có ít nhất 2 phần tử cùng đạt tần số xuất hiện lớn nhất trong đoạn con đó.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Phần tử phổ biến nhất toàn mảng M chắc chắn là một trong hai phần tử đạt tần số lớn nhất trong đoạn con tối ưu.
 - Chia căn dựa trên tần số của các phần tử: Ngưỡng $B \approx \sqrt{N}$.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Với các phần tử có tần số $> B$ (tối đa N/B phần tử): Ghép cặp từng phần tử với M . Quy về bài toán tìm đoạn con có tổng bằng 0 với $A_i = +1$ nếu là M , -1 nếu là phần tử kia, 0 nếu là số khác. Quét tìm kiếm bằng mảng vị trí đầu tiên xuất hiện tổng tiền tố trong $O(N)$.
 - Với các tần số $\leq B$: Duyệt từng tần số $f \in [1, B]$, dùng Two Pointers tìm đoạn con dài nhất có tần số lớn nhất bằng đúng f .
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O(N\sqrt{N})$, Bộ nhớ: $O(N)$.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.